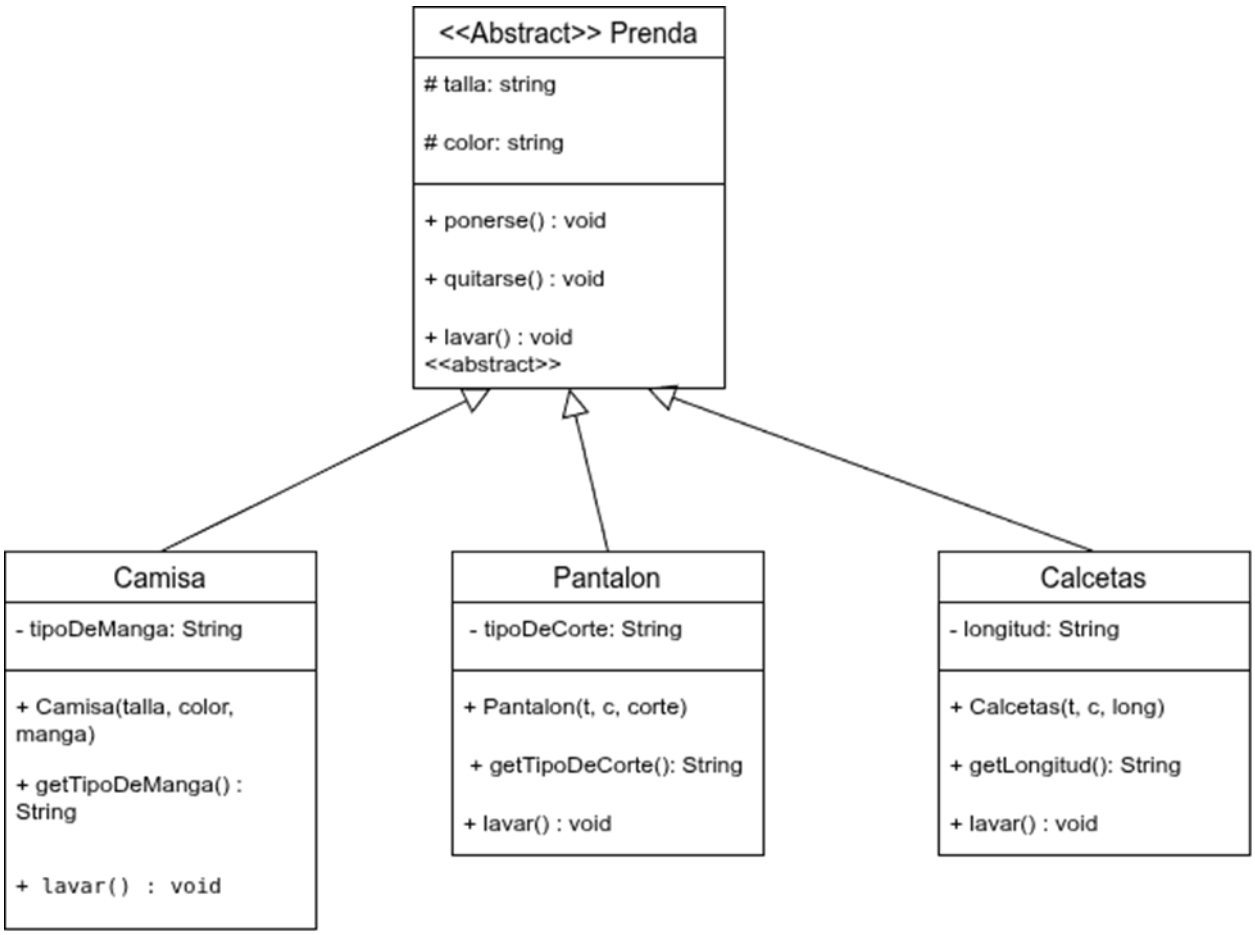
Justification of the class diagram

Garment management system



1. The problem being modeled

The system aims to represent and manage different types of clothing items (shirts, pants, and socks). For any garment, we generally need to know basic shared characteristics, such as its size and color. Furthermore, all garments share common actions: a person can put them on, take them off, or wash them.

However, each specific type of garment has unique characteristics (for example, a shirt has a certain type of sleeve, but pants have a certain type of cut). Likewise, although all garments are washed, the washing process can vary depending on the type of garment.

This class diagram represents exactly that structure: how garments are hierarchically organized, reusing common attributes and defining specific behaviors through inheritance.

2. Why was a class diagram chosen?

Unlike other models, the class diagram shows the data structure (what attributes each entity has). It's the right tool here because the problem describes static entities, their characteristics (size, color), and their capabilities (methods), rather than the order in which events occur. It allows us to visualize key object-oriented programming concepts such as inheritance, encapsulation, and polymorphism.

3. The participants: why these and not others

- **Garment (Parent Class / Superclass): It** is modeled as a central and abstract entity because it groups all common data and behaviors. It would not make sense to repeat the attributes "size" and "color" in each individual garment; centralizing them here optimizes the model.

- **Shirt, Pants, Socks (Child Classes / Subclasses):** These are modeled as separate classes because they represent concrete objects with unique properties. A shirt needs to have its SleeveType recorded, pants its CutType, and socks their Length.

4. Structure and justification of methods and attributes

The following table explains where each element of the diagram comes from and why it has that visibility and location.

Element (Attribute/Method)	**Justification (why it exists and its design)**	**Notation used**
size, color (in Garment)	These are universal characteristics of any clothing. They are placed in the superclass so that subclasses automatically inherit them without rewriting code.	Protected attributes (#). This allows child classes to access them directly, but they remain hidden from the rest of the system.
put on(), take off() (on Garment)	These are generic actions that apply equally to any clothing item. Their implementation does not change, so they are specifically defined in the parent code.	Public methods (+), indicating that any actor or part of the system can invoke these actions.
wash() (on Garment)	All clothes get dirty and need to be washed, so laundry is essential. However, it's not possible to define how *to* wash them generically (a silk shirt is washed differently than cotton socks).	Abstract method (<<abstract>> and/or italics) and public (+). It forces child classes to program their own version of the wash.
sleeveType, CutType, Length	These are unique attributes. A shirt doesn't have "length" in the same way that socks do, nor do socks have a "cut type."	Private attributes (-). They reinforce encapsulation; no one external can modify them directly.
Shirt(...), Trousers(...), Socks(...)	These are the constructor methods. They are used to create (instantiate) objects when the system starts, receiving the necessary data (size, color, and the specific attribute).	Public construction methods (+).

getTipoDeManga(), etc.	Since the specific attributes are private (-), the system needs a secure way to query or read them without altering them.	Public "getter" methods (+) that return a String type.
wash() (in subclasses)	It is the mandatory response to the father's abstract method. Each garment provides its own specific washing instructions (polymorphism).	Public methods (+).

5. Conclusión: The diagram is easy to understand once you know about "parent" classes—known as abstract classes—and the subsequent classes that inherit attributes from them. Attributes represent the characteristics an object possesses, while methods are the actions it can perform. There is also polymorphism, where a method might share the same name yet execute two different actions.